

Callbacks & Schedulers

1. Callback function

A Callback is a function that is passed as an argument to another function.

1.1 Passing a function as an argument

```
function displayGreeting(displayName) {  
  console.log("Hello");  
  displayName();  
  console.log("Good Morning!");  
}
```

```
displayGreeting(function() {  
  console.log("Rahul");  
});
```

1.2 Passing a function name as an argument

```
function displayGreeting(displayName) {  
  console.log("Hello");  
  displayName();  
  console.log("Good Morning!");  
}  
function displayRahul() {  
  console.log("Rahul");  
}  
displayGreeting(displayRahul);
```

1.3 Passing a function expression as an argument

```
function displayGreeting(displayName) {  
  console.log("Hello");  
  displayName();  
  console.log("Good Morning!");  
}
```

```
}  
let displayRam = function() {  
  console.log("Ram");  
};  
displayGreeting(displayRam);
```

2. Schedulers

The Schedulers are used to schedule the execution of a callback function.

There are different scheduler methods.

- setInterval()
- clearInterval()
- setTimeout()
- clearTimeout(), etc.

2.1 setInterval()

The

setInterval() method allows us to run a function at the specified interval of time repeatedly.

Syntax:

```
setInterval(function, delay);
```

function - a callback function that is called repeatedly at the specified interval of time (

delay). **delay** - time in milliseconds. (1 second = 1000 milliseconds)

```
let counter = 0;  
setInterval(function() {  
  console.log(counter);  
  counter = counter+1;  
}, 1000);  
In the
```

`setInterval()` method, the callback function repeatedly executes until the browser tab is closed or the scheduler is cancelled.

When we call the

`setInterval()` method, it returns a unique id. This unique Id is used to cancel the callback function execution.

2.2 clearInterval()

The

`clearInterval()` method cancels a schedule previously set up by calling `setInterval()`.
To execute

`clearInterval()` method, we need to pass the `uniqueId` returned by `setInterval()` as an argument.

Syntax:

```
clearInterval(uniqueId);  
let counter = 0;  
let uniqueId = setInterval(function() {  
  console.log(counter);  
  counter = counter+1;  
}, 1000);
```

```
clearInterval(uniqueId);
```

2.3 setTimeout()

The

`setTimeout()` method executes a function after the specified time.

Syntax:

```
setTimeout(function, delay);
```

function - a callback function that is called after the specified time (

delay). **delay** - time in milliseconds.

```
let counter = 0;
setTimeout(function() {
  console.log(counter);
  counter = counter + 1;
}, 1000);
```

2.4 clearTimeout()

We can cancel the

setTimeout() before it executes the callback function using the clearTimeout() method.
To execute

clearTimeout(), we need to pass the uniqueId returned by setTimeout() as an argument.

Syntax:

```
clearTimeout(uniqueId);
let counter = 0;
let uniqueId = setTimeout(function() {
  console.log(counter);
  counter = counter+1;
}, 1000);

clearTimeout(uniqueId);
```